📚 Estudo de Caso: O Compilador CPython

> **Disciplina:** Teoria da Computação e Compiladores — UNIBH 2026/1
> **Professora:** Charlene Cássia de Resende
> **Aluno:** Sérgio Pinton Pavanelli | **RA:** 123220202
> **Entregável 1:** Estudo de caso sobre um compilador existente

🎯 Objetivos deste documento

- Compreender a arquitetura interna do compilador CPython
- Identificar as fases clássicas de compilação no mundo real
- Comparar CPython com outras ferramentas modernas (GCC, LLVM)
- Extrair lições que fundamentam o protótipo TinyScript

1. 🐍 Por que CPython?

O **CPython** é a implementação de referência da linguagem Python, escrita em C e disponível como software livre (licença PSF). Trata-se de um compilador + máquina virtual integrados: o código-fonte Python é compilado para **bytecode** e executado pela **CPython Virtual Machine (CPython VM)**.

Critério	Valor
Linguagem de implementação	C
Versão de referência	Python 3.13 (2024)
Repositório oficial	github.com/python/cpython
Licença	Python Software Foundation License
Paradigma	Compilador + Interpretador híbrido

2. 🏗️ Arquitetura Geral

O CPython processa o código-fonte em cinco etapas distintas:

```
Código-fonte (.py)
        │
        ▼
┌──────────────────┐
│  1. Tokenização  │  módulo: tokenize.py
```

```
│  (Análise Léxica) │  → produz stream de tokens
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  2. Parsing      │  módulo: Grammar/Grammar (PEG parser)
│  (Análise Sintática│  → produz CST (Concrete Syntax Tree)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  3. AST Building │  módulo: ast.py / Python.asdl
│                  │  → produz AST (Abstract Syntax Tree)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  4. Compilação   │  módulo: compile.c / symtable.c
│  para Bytecode   │  → produz .pyc (bytecode + metadados)
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  5. Execução     │  módulo: ceval.c (loop principal)
│  pela CPython VM │  → pilha de execução (stack machine)
└──────────────────┘
```

3. 🔍 Fase 1 — Análise Léxica (Tokenização)

Como funciona no CPython

O módulo `tokenize` (Python puro) e o tokenizador em C (`Parser/tokenize.c`) percorrem o arquivo `.py` caractere a caractere e produzem uma sequência de tokens.

Tipos de tokens em Python:

Tipo	Exemplos
`NAME`	`if` , `while` , `x` , `resultado`
`NUMBER`	`42` , `3.14` , `0xFF`
`STRING`	`"hello"` , `'world'` , `"""multi"""`
`OP`	`+` , `-` , `=` , `==` , `(` , `)`
`NEWLINE` / `INDENT` / `DEDENT`	controle de indentação
`COMMENT`	`# comentário`

Tipo	Exemplos
`ENDMARKER`	fim do arquivo

Particularidade do Python: A indentação é semanticamente significativa — os tokens `INDENT` e `DEDENT` substituem `{` e `}` de outras linguagens. Isso exige que o lexer mantenha uma **pilha de níveis de indentação**.

Inspeção prática (Python 3):

```
import tokenize, io
codigo = "x = 10 + y"
tokens = list(tokenize.generate_tokens(io.StringIO(codigo).readline))
# Saída:
# TokenInfo(type=1 'NAME', string='x', ...)
# TokenInfo(type=54 'OP', string='=', ...)
# TokenInfo(type=2 'NUMBER', string='10', ...)
# TokenInfo(type=54 'OP', string='+', ...)
# TokenInfo(type=1 'NAME', string='y', ...)
```

4. 🌳 Fase 2 e 3 — Análise Sintática e AST

Parser PEG (desde Python 3.9)

Até o Python 3.8, o CPython usava um parser LL(1) baseado em gramática BNF. A partir do Python 3.9, foi migrado para um **PEG Parser** (Parsing Expression Grammar), gerado pela ferramenta `pegen`.

Vantagens do PEG:

- Suporta gramáticas mais expressivas (sem ambiguidades)
- Permite uso de lookahead ilimitado com memoização
- Gramática descrita em `Grammar/python.gram`

AST do CPython:

A AST é descrita formalmente no arquivo `Parser/Python.asdl` (Abstract Syntax Description Language). Cada construção da linguagem tem um nó correspondente.

Exemplo: `x = 10 + y` gera:

```
Module(body=[
  Assign(
    targets=[Name(id='x', ctx=Store())],
    value=BinOp(
      left=Constant(value=10),
      op=Add(),
```

```
      right=Name(id='y', ctx=Load())
    )
  )
])
```

Inspeção prática:

```python
import ast
tree = ast.parse("x = 10 + y")
print(ast.dump(tree, indent=2))
```

5. ⚙️ Fase 4 — Geração de Bytecode

O compilador CPython (`Python/compile.c`) transforma a AST em **bytecode** — uma sequência de instruções para a máquina virtual CPython (stack machine de 16 bits por instrução).

Principais instruções bytecode:

Instrução	Descrição
`LOAD_CONST`	Empilha uma constante
`LOAD_FAST`	Empilha variável local
`STORE_FAST`	Desempilha e armazena em variável
`BINARY_OP`	Operação aritmética (topo da pilha)
`COMPARE_OP`	Comparação
`POP_JUMP_IF_FALSE`	Desvio condicional
`CALL`	Chama uma função
`RETURN_VALUE`	Retorna valor

Inspeção prática:

```python
import dis
codigo = """
x = 10 + 5
if x > 12:
    print(x)
"""
dis.dis(codigo)
# Saída (trecho):
#   LOAD_CONST  10
#   LOAD_CONST  5
#   BINARY_OP   0 (+)
```

```
#   STORE_NAME  x
#   ...
```

Os arquivos `.pyc` armazenam o bytecode com cabeçalho de versão e timestamp para cache.

6. 🚀 Fase 5 — Execução pela CPython VM

O arquivo `Python/ceval.c` contém o **loop de avaliação** — um `switch` enorme que despacha cada opcode. A máquina virtual CPython usa uma **pilha de operandos** (stack machine) e mantém um quadro de execução (`PyFrameObject`) por chamada de função.

Otimizações em Python 3.12+:

- **Specializing Adaptive Interpreter**: instruções são especializadas em runtime com base nos tipos observados (ex.: `BINARY_OP` → `BINARY_OP_ADD_INT`)
- Redução de overhead de despacho com `computed gotos` (GCC)

7. ⚖️ Comparativo: CPython × GCC × LLVM

Característica	CPython	GCC	LLVM/Clang
Paradigma	Compilador + Interpretador	Compilador nativo	Framework de compilação
Representação intermediária	Bytecode (stack-based)	GIMPLE / RTL	LLVM IR (SSA form)
Geração de código	Bytecode → VM	Binário nativo	Binário nativo / JIT
Otimizações	Limitadas (peephole)	Agressivas (-O3)	Modulares por pass
Portabilidade	Alta (VM universal)	Depende do backend	Alta (múltiplos backends)
Casos de uso	Scripts, IA, web	Sistemas, embarcados	Compiladores modernos, WASM
Parser	PEG (pegen)	Recursivo descendente	Recursivo descendente
Front-end	tokenize + pegen	cpp + cc1	clang

CPython vs. LLVM — diferença fundamental:

> O CPython compila para uma VM portável (compromisso entre velocidade e portabilidade). O LLVM compila para código de máquina altamente otimizado via IR em forma SSA, com análise de fluxo de dados entre passes de otimização.

8. 💪 Pontos Fortes e Limitações do CPython

✅ Pontos Fortes

- **Portabilidade:** o mesmo bytecode executa em qualquer plataforma com CPython instalado
- **Introspecção:** `ast`, `dis`, `tokenize` permitem análise e transformação de código em runtime
- **Ecossistema:** integração com C/C++ via CPython API; base para PyPy, Cython, Numba
- **Legibilidade da gramática:** `Grammar/python.gram` é humano-legível

⚠️ Limitações

- **GIL (Global Interpreter Lock):** impede paralelismo real de threads (mitigado no Python 3.13 com modo free-threaded experimental)
- **Performance:** significativamente mais lento que código compilado nativamente (GCC/LLVM)
- **Sem tipos estáticos:** ausência de inferência de tipos dificulta otimizações agressivas (PyPy usa JIT tracing como solução)

9. 📖 Lições para o Protótipo TinyScript

Lição do CPython	Aplicação no TinyScript
Separação clara de fases (tokenizer → parser → AST → codegen)	Módulos `lexer.py`, `parser.py`, `ast_nodes.py`, `codegen.py`
AST como representação intermediária universal	Classes de nós em `ast_nodes.py`
Parser recursivo descendente (simples e legível)	`parser.py` usa descent recursivo
Código intermediário antes do alvo final	Geração de TAC (Three-Address Code) em `codegen.py`
Tokens com informação de posição (linha/coluna)	`Token(tipo, valor, linha, coluna)`

📚 Referências

1. **AHO, A. V.; LAN, M. S.; SETHI, R.; ULLMAN, J. D.** Compiladores: princípios, técnicas e ferramentas. 2. ed. Pearson, 2008. *(Dragon Book)*
2. **CPython Source Code.** Disponível em: https://github.com/python/cpython
3. **Python Language Reference — Internal Compiler.** Disponível em: https://devguide.python.org/internals/compiler/
4. **MENDEZ, P.; BLAUTH, P.** Linguagens formais e autômatos. 6. ed. Bookman, 2010.
5. **SANTOS, P. R.; LANGLOIS, T.** Compiladores: da teoria à prática. LTC, 2019.